Question 1: Detecting and Exploiting SQL Injection Vulnerabilities

Scenario:

You are an IT auditor tasked with assessing the security of a web application running on a Linux server. The application interfaces with a MySQL database. During your assessment, you identify a URL endpoint that includes a query parameter:

```
http://example.com/view.php?id=10
```

Tasks:

1. Explain how you would test this endpoint for SQL injection vulnerabilities using CLI-based tools.
2. If a vulnerability is found, describe the steps to exploit it to enumerate the database's table names.
3. Recommend mitigation strategies to prevent SQL injection attacks.

Solution:

1. **Testing for SQL Injection:**
   - **Manual Testing:** Use `curl` to send HTTP requests with payloads designed to test for SQL injection. For example:
     ```
     curl "http://example.com/view.php?id=10'"
     ```
     If the application returns a database error or behaves unexpectedly, it may be vulnerable to SQL injection.
   - **Automated Testing:** Utilize tools like `sqlmap` to automate the detection process. First, capture the request using a proxy tool and save it to a file (e.g., `request.txt`). Then run:
     ```
     sqlmap -r request.txt --dbms=mysql --batch --level=2
     ```
     This command tests for SQL injection vulnerabilities targeting MySQL databases.
2. **Exploiting the Vulnerability:**
   If `sqlmap` confirms the presence of a SQL injection vulnerability, you can enumerate the database's table names with:
   ```
   sqlmap -r request.txt --dbms=mysql --tables
   ```
   This command retrieves all table names from the database, aiding in further assessment of the database structure and potential data exposure.
3. **Mitigation Strategies:**
   - **Use Prepared Statements:** Implement parameterized queries to ensure user inputs are treated as data, not executable code.
   - **Input Validation:** Validate and sanitize all user inputs to reject malicious data.
   - **Least Privilege Principle:** Ensure the database user account has the minimum necessary permissions.
   - **Error Handling:** Configure the application to display generic error messages, avoiding the exposure of database details.
   - **Regular Security Testing:** Conduct periodic security assessments to identify and remediate vulnerabilities promptly.

Question 2: Identifying and Mitigating Cross-Site Scripting (XSS) Vulnerabilities

Scenario:

During a security audit of a web application hosted on a Linux server, you suspect that the application is vulnerable to Cross-Site Scripting (XSS) attacks. The application reflects user input from the `search` query parameter in the response without proper sanitization.

Tasks:

1. Demonstrate how to test for XSS vulnerabilities using CLI-based tools.
2. Explain the potential impacts of an XSS vulnerability in this context.
3. Suggest remediation measures to protect against XSS attacks.

Solution:

1. **Testing for XSS:**
   - **Manual Testing:** Use `curl` to inject a simple script into the `search` parameter:
     `curl "http://example.com/search.php?query=<script>alert('XSS')</script>"`
     If the injected script executes in the user's browser, the application is vulnerable to XSS.
   - **Automated Testing:** Employ CLI-based security scanners like the Web Application Security Scanner to detect XSS vulnerabilities.
2. **Potential Impacts:**
   - **Session Hijacking:** Attackers can steal cookies and impersonate users.
   - **Defacement:** Malicious scripts can alter the appearance of the website.
   - **Data Theft:** Sensitive information displayed on the page can be exfiltrated.
   - **Malware Distribution:** Users can be redirected to malicious sites or have malware delivered via the vulnerable application.
3. **Remediation Measures:**
   - **Input Sanitization:** Encode or strip out HTML tags and JavaScript from user inputs.
   - **Content Security Policy (CSP):** Implement CSP headers to restrict the sources from which scripts can be executed.
   - **Output Encoding:** Ensure that data output to the browser is properly encoded to prevent script execution.
   - **Regular Code Reviews:** Conduct thorough reviews to identify and fix improper handling of user inputs.

Question 3: Assessing and Securing Session Management

Scenario:

As part of an IT audit, you are evaluating the session management mechanisms of a web application running on a Linux platform. You discover that session tokens are predictable, and session IDs are included in the URL.

Tasks:

1. Describe how to test the predictability of session tokens using CLI tools.
2. Explain the security risks associated with predictable session tokens and session IDs in URLs.
3. Recommend best practices for secure session management.

Solution:

1. Testing Session Token Predictability Using CLI Tools

To assess whether session tokens are predictable, we can collect multiple tokens and analyze their patterns.

- **Collecting Session Tokens:** Use `curl` to repeatedly log in or request session tokens and extract them from responses:
  `for i in {1..100}; do curl -i "http://example.com/login" | grep "Set-Cookie"; done`
  If the session ID follows a predictable pattern (e.g., sequential or timestamp-based), it is a serious security flaw.
- **Analyzing Token Patterns:** Store the session tokens in a file and check for sequences or similarities:
  `cat tokens.txt | sort | uniq -c`
  If the same token repeats or follows an incremental pattern, the session mechanism lacks sufficient randomness.
- **Entropy Testing Using Python:** To quantify randomness, use a Python script to compute Shannon entropy:

```
import math
from collections import Counter

def shannon_entropy(data):
    freq = Counter(data)
    total = len(data)
    return -sum((count/total) * math.log2(count/total) for count in freq.values())

with open("tokens.txt") as f:
```

```
tokens = f.read().splitlines()
print("Entropy:", shannon_entropy(''.join(tokens)))
```

A low entropy score suggests predictability.

---

2. Security Risks of Predictable Session Tokens and Session IDs in URLs

- **Session Hijacking:** If attackers can predict session tokens, they can impersonate users and gain unauthorized access.
- **Replay Attacks:** Predictable session tokens allow replaying requests, leading to potential unauthorized transactions.
- **URL-Based Session Fixation:** When session IDs are exposed in URLs (e.g., `http://example.com/dashboard?session=abc123`), they can be leaked via:
  - Browser history
  - Referrer headers
  - Log files
  - Shared links
- **Privilege Escalation:** If session tokens follow a sequential pattern (e.g., `session123`, `session124`), attackers can guess higher-privileged accounts.

---

3. Best Practices for Secure Session Management

- **Use Cryptographically Secure Tokens:**
  - Generate session tokens using a secure random function (e.g., OpenSSL):
    `openssl rand -hex 32`
  - Avoid sequential or timestamp-based session IDs.
- **Implement Secure Cookie Flags:**
  - Set `HttpOnly` to prevent JavaScript access:
    `Set-Cookie: session=xyz123; HttpOnly`
  - Enable `Secure` flag to restrict transmission to HTTPS:
    `Set-Cookie: session=xyz123; Secure`
- **Prevent Session Fixation:**
  - Regenerate session IDs upon login:
    `session_regenerate_id(true);`
- **Avoid Exposing Session IDs in URLs:**
  - Use cookies instead of URL parameters.
  - Implement session expiration and automatic logout.
- **Monitor and Log Anomalous Session Activity:**
  - Detect multiple logins with the same token.
  - Alert admins on suspicious token patterns.

By implementing these best practices, the security risks associated with weak session management can be mitigated effectively.